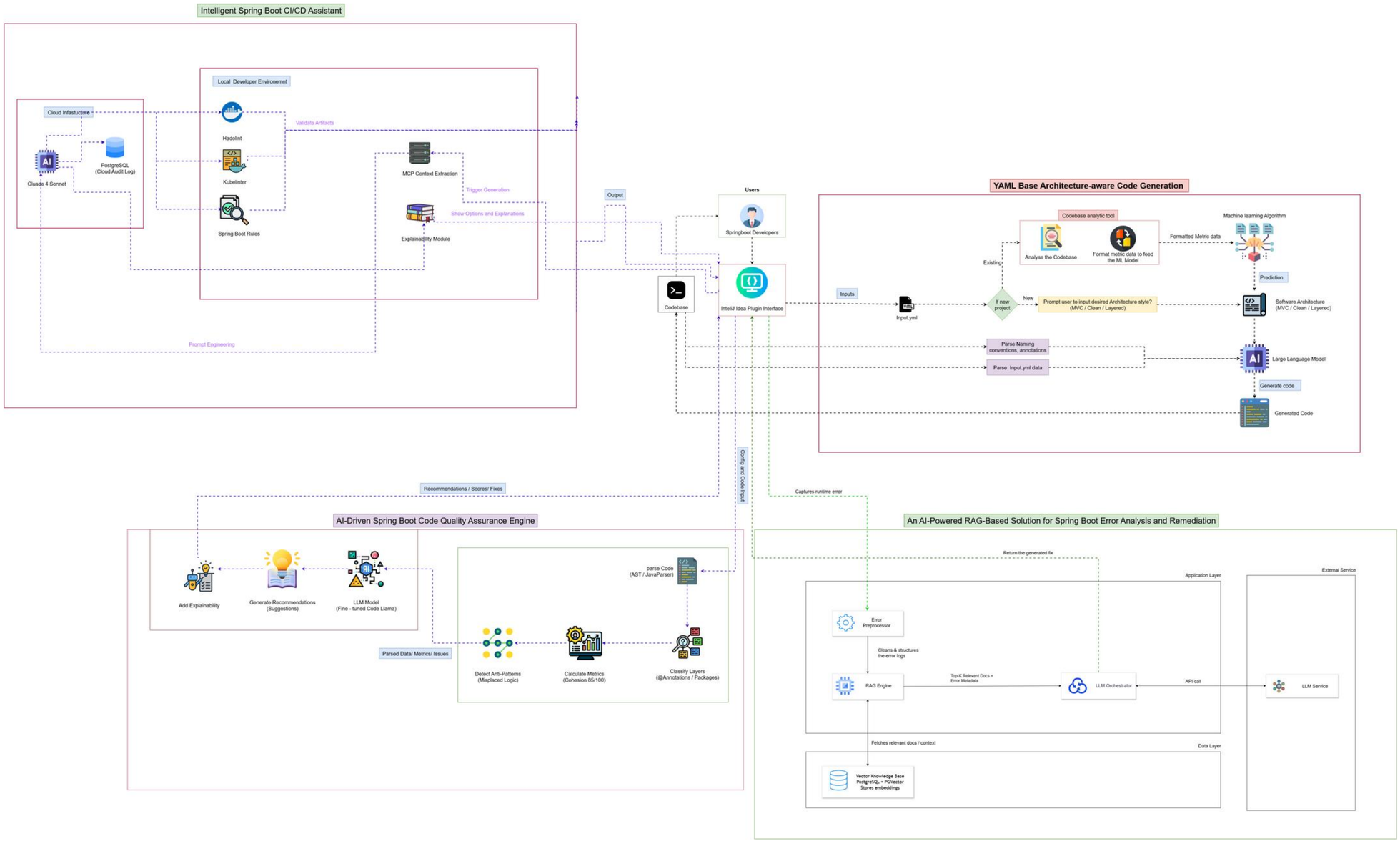
SpringForge

Intelligent Spring Boot Development Ecosystem

System Overview

Introdu

Overall System Diagram



IT22076052

Mahindarathna D.G.P.T.D

Software Engineering

25-26J-
451

ARCHITECTURE-AWARE Spring Boot Code Generation Plugin

Architecture-Aware Spring Boot Code Generation Plugin

Problem

Spring Boot dominates enterprise backends, but existing code generation (UML, LLMs) fails to enforce its conventions. No IDE-native, architecture-aware generator exists. Our IntelliJ plugin combines static analysis, ML, and LLMs to generate consistent, convention-compliant Spring Boot code inside the IDE.

Proven

No current system provides Spring Boot–specific, architecture-aware code generation.
Proposed: an IntelliJ IDEA plugin that combines static analysis + ML-based architecture prediction + LLM-driven generation, taking YAML inputs and producing project-specific, architecture-consistent Spring Boot code seamlessly inside the IDE.

Existing Tools

Features	GitHub Copilot	JHipster	Spring Initializr	Telosys Tool	Proposed IntelliJ Plugin
Code Generation Method	AI-powered real-time completion	Template-driven full-stack scaffolding	Configuration-based project bootstrapping	Model-driven code generation	YAML-specified ML-enhanced architecture generation
Architecture Understanding	Pattern-agnostic context-limited	Microservices template-based	No awareness	Template dependent	Pattern detection architectural prediction
Development Environment	Multi-IDE native support	CLI + IDE import workflow	Web-based with IDE import	Eclipse/IDEA plugin support	Native IntelliJ integration
Customization	Prompt tuning	Extensive templates	Fixed options	Template creation	YAML configuration
Output Quality	Context-dependent variable accuracy	Production-ready rigid structure	Basic boilerplate only	Template-quality dependent	Architecture-consistent validated code

Architecture-Aware Spring Boot Code Generation Plugin

Key domains

- **Software Architecture & Design Patterns** → enforcing Layered, MVC, Clean styles.
- **Machine Learning & Static Analysis**→ predicting architecture from code features.
- **Software Engineering Practices** →Agile SDLC, plugin engineering, IDE-native integration.

Latest Technologies

- **Machine Learning:** Python, Scikit-learn, TensorFlow (Random Forest, XGBoost, Neural Networks)
- **LLM Frameworks:** Anthropic's Claude4(Bedrock) for code generation
- **Tools & Frameworks:** Java, Spring Boot, IntelliJ IDEA SDK

Evaluation &

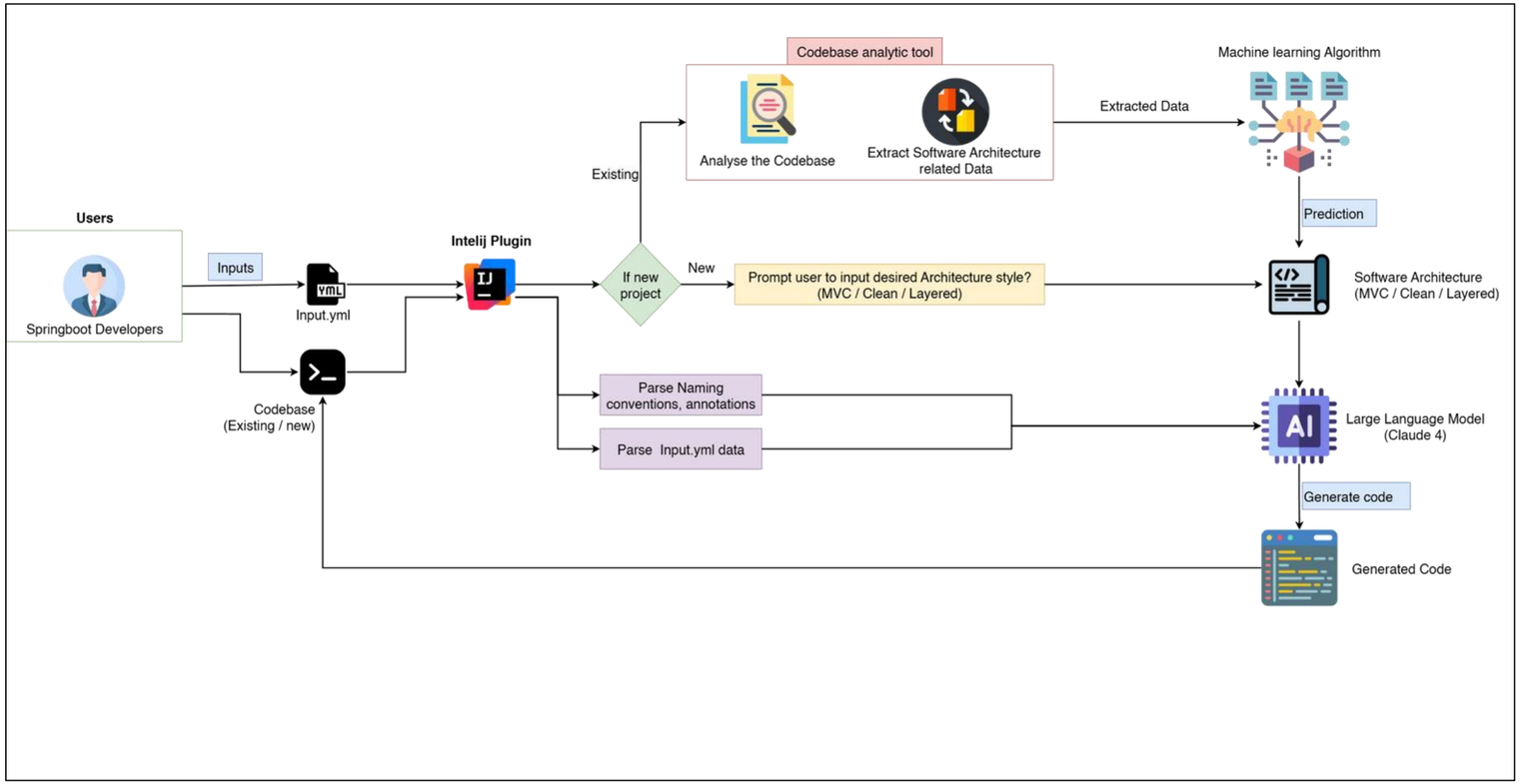
- Architecture Prediction Accuracy ≥75% on curated datasets.
- Compilation Success Rate of generated code in real projects using developer feedback.

Data Availability and

- **Training data:** Open-source Spring Boot projects from GitHub (publicly available).
- No personal/sensitive data used → no ethical risks.
- Manual curation ensures dataset quality & transparency.

Architecture-Aware Spring Boot Code Generation Plugin

System



real-world

Scenario: A development team starting a new Spring Boot project with MVC architecture needs to scaffold entities, repositories, and controllers that align with their project's architecture.

- Developer provides a **YAML file** describing entities and relationships in IntelliJ.
- **Plugin** scans the existing project (or new setup) to capture conventions and patterns.
- **Static Analysis Module** extracts architecture-related features such as package structure, annotations, and dependencies.
- **Classification Model** identifies the architectural style (Layered, MVC, Clean)
- System Feed the project-specific conventions, classified architecture and other details to the LLM.
- **LLM Generator (Claude 4)** produces architecture-consistent, convention-compliant Spring Boot code directly into the project structure.

IT22562524

Ariyasena M.K.M.M.
Software Engineering
25-26J-
451

An AI-Powered RAG-Based Solution for Spring Boot Runtime Error Analysis and Remediation

An AI-Powered RAG-Based Solution for Spring Boot Runtime Error Analysis and Remediation

Problem

Debugging runtime errors in frameworks like Spring Boot is time-consuming; developers rely on manual log analysis & searching external resources. Existing AI assistants provide explanations but are generic / outdated since they lack retrieval grounding.

Proven

No current system combines runtime error capture + RAG. Proposed: IDE plugin with RAG backend captures runtime errors, retrieves Spring Boot documentations / Stack overflow threads / GitHub fixes, and generates accurate explanations and remedations.

Features	Generic AI Chatbots	GitHub Copilot	JetBrains AI	Proposed Feature
Runtime Error Capture	✗	✗	✓	✓
RAG Support	✗	✓	✗	✓
Accuracy for Frameworks	Generic	Moderate	Generic	Springboot aware
Fix Suggestions	Limited	Limited	Basic	Automated, verified

IT22562524 | Ariyasena M.K.M.M. | 25-26J-451

An AI-Powered RAG-Based Solution for Spring Boot Runtime Error Analysis and Remediation

Key Domains

- Software Debugging & Program Analysis
- Large Language Models (LLMs)
- Retrieval-Augmented Generation (RAG)
- Spring Boot Framework

Latest Technologies

- **IntelliJ Plugin + Backend:** Java
- **RAG Pipeline:** Python
- **Knowledge base:** Postgres + pgvector
- **Embeddings:** all-MiniLM-L6-v2
- **LLM Model:** Claude Sonnet 4

An AI-Powered RAG-Based Solution for Spring Boot Runtime Error Analysis and Remediation

How to

- Compare assistant's debugging accuracy and relevance of suggestions with traditional debuggers or manual search.
- Measure % of runtime errors where the system provides a correct and actionable fix (target ≥80%).
- Validate latency from error capture → suggestion (target ≤5 seconds).
- Conduct user surveys/interviews to assess clarity, usefulness, and trust in suggestions.
- Track time saved in resolving errors compared to manual debugging.

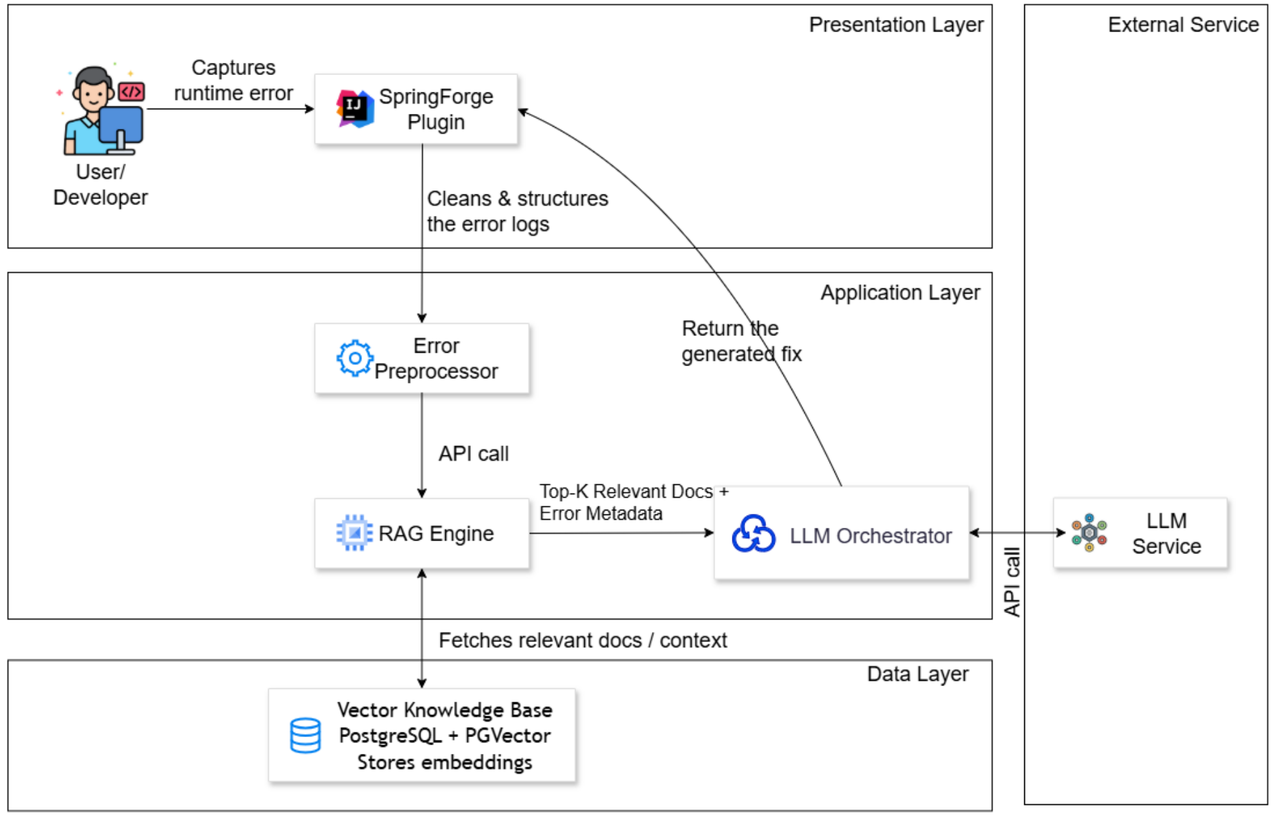
Data

- Public Stack Overflow threads
- GitHub issues
- Spring Boot docs

Ethical

- Respect content licenses of public sources
- No personal/sensitive data used
- Anonymize user/project data

An AI-Powered RAG-Based Solution for Spring Boot Runtime Error Analysis and Remediation



Scenario: A developer working on a Spring Boot application encounters recurring runtime errors during integration testing.

1.Developer encounters runtime error in their IDE
2.SpringForge Plugin automatically captures error details and context
3.Error Preprocessor cleans and structures the error information
4.RAG Engine retrieves relevant documentation and similar error patterns
5.LLM Orchestrator generates contextual fix suggestions with explanations

IT22060662

Jameela M.J.F.

Software Engineering

25-26J-451

AI-Driven Code Quality Spring Boot Assurance Engine As A Plugin

AI-Driven Code Quality Spring Boot Assurance Engine As A Plugin

Problem

- Maintaining Spring Boot code quality is challenging due to lack of real-time, architecture-aware analysis. Generic tools miss 75-80% of anti-patterns, increasing debugging time by 40%.IDE solutions lack precise, layer-specific feedback, including Anti-Pattern Classification Model and Code Quality Score Regression Model, affecting 70%+ projects' quality .

Proven Gap

No current system integrates real-time,architecture-aware analysis with ML-driven Anti-Pattern Classification and Code Quality Score Regression. Proposed: IDE plugin leverages these models, detects Spring Boot-specific issues, and provides precise, layer-specific recommendations using fine-tuned LLM and RAG .

Tool	Spring Boot Awareness	Architecture Pattern Support	AI-Based Recommendations	Anti-Pattern Classification Model	Code Quality Score Regression Model	Real-time IDE Integration
SonarQube	✓ (Basic rules)	✗	✗	✗	✗	✓ (Limited)
Qodana	✗	✗	✓ (Generic)	✗	✗	✓
CodeScene	✗	✗	✓ (Context trends)	✗	✗	✓
Proposed Engine	✓ (Deep, layer-specific)	✓ (User-configured)	✓ (Spring Boot optimized)	✓ (ML-driven, 90%+ accuracy)	✓ (Layer-specific regression)	✓ (Full)

Key domains

- Spring Boot Framework & ArchitecturalPatterns
- → Layered, MVC, Clean Architectures; layer-specific analysis (Controller, Service,Repository, Entity).
- Machine Learning & Static Code Analysis
- →Anti-Pattern Classification and CodeQuality Score Regression Model
- Large Language Models (LLMs) & NLP
- Retrieval-Augmented Generation (RAG)
- Software Engineering Practices

Latest

- Machine Learning: Python, Scikit-learn (SVM for Anti-Pattern Classification Model),Gradient Boosting (Code Quality Score Regression Model)
- LLM Frameworks: Fine-tuned Code Llama for Recommendations
- Tools & Frameworks: Java, Spring Boot,IntelliJ IDEA SDK, JavaParser for AST Analysis

How to Evaluate

- Benchmark anti-pattern detection and qualityscores against baseline tools, measure developer satisfaction via time saved and surveys.
- Fix Accuracy: Percentage of suggested fixesresolving anti-patterns.
- Anti-Pattern Classification Model Accuracy:Precision in detecting issues (e.g., business logic inControllers).
- Code Quality Score Regression Model Precision:Accuracy of layer-specific metric predictions.
- Recommendation Relevance: Effectiveness of LLM-generated suggestions.

Data Sources:

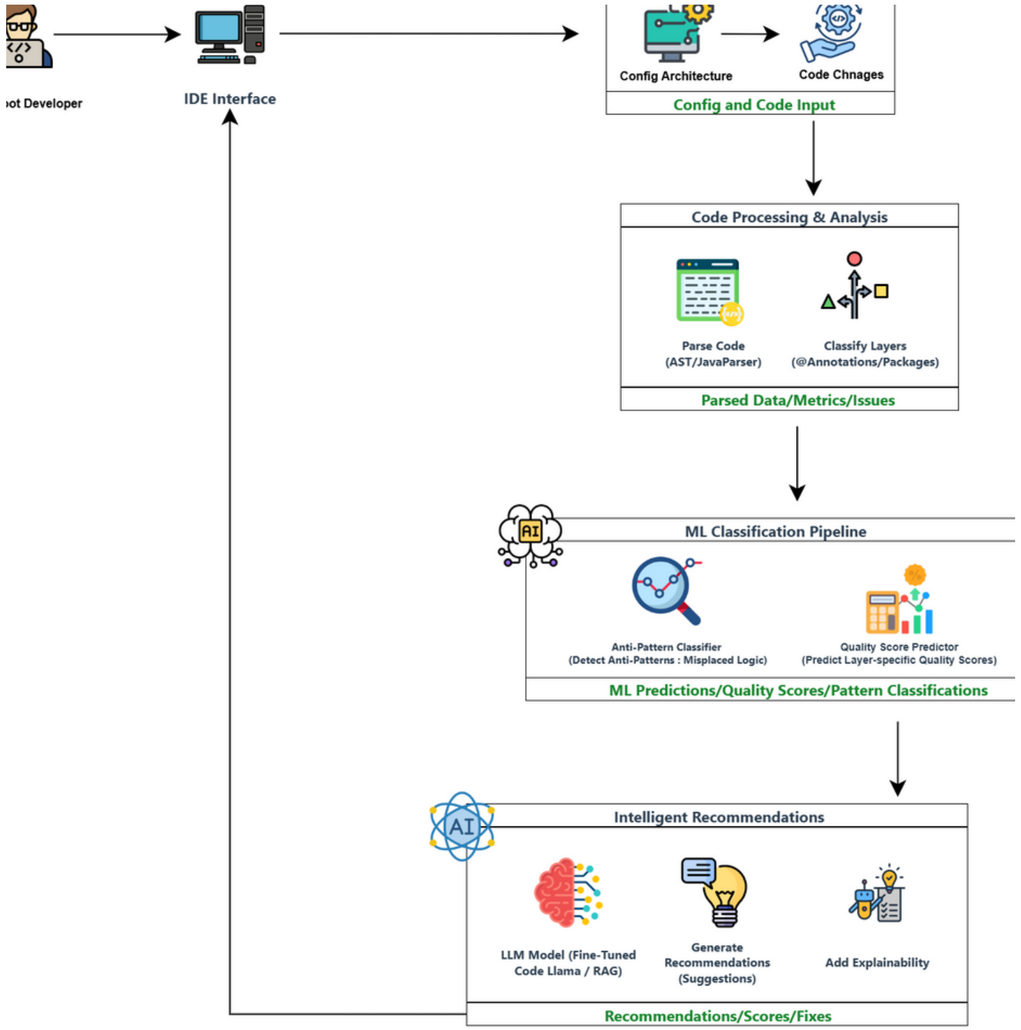
- Training data: Open-source Spring Boot projects from GitHub (publicly available).
- Public Stack Overflow threads, GitHub issues, Spring Boot documentation for model training and recommendations.

Ethical Considerations:

- Respect content licenses of public sources
- No personal/sensitive data used
- Anonymize user/project data

AI-Driven Code Quality Spring Boot Assurance Engine As A Plugin

System



Real-world

Scenario: A development team working on a large Spring Boot application seeks to enhance code quality and maintain architectural consistency during development, identified within their IDE.

1. Developer provides the project architecture (e.g.,Layered) via an IDE setup wizard.
2. SpringForge Plugin scans the project to capture codecontext and conventions.
3. Error Preprocessor analyzes code; Anti-PatternClassification Model detects issues (e.g., business logic inControllers); Code Quality Score Regression Modelcomputes layer-specific metrics.
4. RAG Engine retrieves relevant Spring Bootdocumentation and quality-focused fixes that address specific layer weaknesses, enhancing code maintainability and developer trust.
5. LLM generates contextual, architecture-compliant quality-focused fixes that address specific layer weaknesses, suggestions with explanations tied to Spring Boot best practices, enhanced by ML-driven analysis and contextual data retrieval.

IT22056320

Senevirathna K.M.U.T

Software Engineering

25-26J-451

Intelligent Spring Boot CI/CD Assistant

AI-Powered Infrastructure Generation with Security Validation and Explainability

Problem

- Writing Docker/K8s/CI pipelines for Spring Boot is complex and error-prone. Developers often miss security best practices (non-root, limits, secrets).AI tools are generic and lack Spring Boot context, validation, and explanations. No IDE-integrated tool exists for secure, explainable CI/CD generation.

Proven Gap

- No existing solution can auto-generate Spring Boot CI/CD pipelines inside the IDE with both security validation and explainable outputs.
- Current AI assistants provide generic infra code but lack project-specific context, multi-layer validation, and educational guidance.

Feature	Our Tool	GitHub Copilot	JetBrains AI	CodeWhisperer
SB Context Infra	✓ MCP Analysis	✗ Generic Code	✗ Basic Spring	✗ AWS Generic
Security + AI	✓ 3-Tool + Claude	✗ No Validation	✗ Limited Sec	✗ Basic Scan
Educational	✓ Line-by-Line	✗ Minimal Info	✗ Comments Only	✗ No Learning

AI-Powered Infrastructure Generation with Security Validation and Explainability

Key domains

- DevSecOps & Program Analysis → secure infrastructure and validation
- Large Language Models (LLMs) & Prompt Engineering → AI-powered generation
- Explainable AI → developer learning and trust
- Spring Boot Framework → Java-based enterprise apps and pipelines

Latest

- Java → IntelliJ Plugin development
- PostgreSQL → audit logs and usage analytics
- Claude 4 Sonnet (Bedrock) → LLM for generation & explanations
- Hadolint & KubeLinter → Docker & Kubernetes security validation
- Model Context Protocol (MCP) → live project metadata extraction

How to Evaluate

- Time Efficiency → Reduction in developer time spent writing CI/CD pipelines.
- Security Compliance → Number/severity ofissues caught by Hadolint, KubeLinter, SpringBoot Rules.
- Artifact Accuracy → % of generated Docker/K8s pipelines that run successfully without manual fixes.
- Explainability Quality → Developer survey: clarity and usefulness of AI explanations.
- User Satisfaction → Surveys/interviews on usability and trust in outputs.

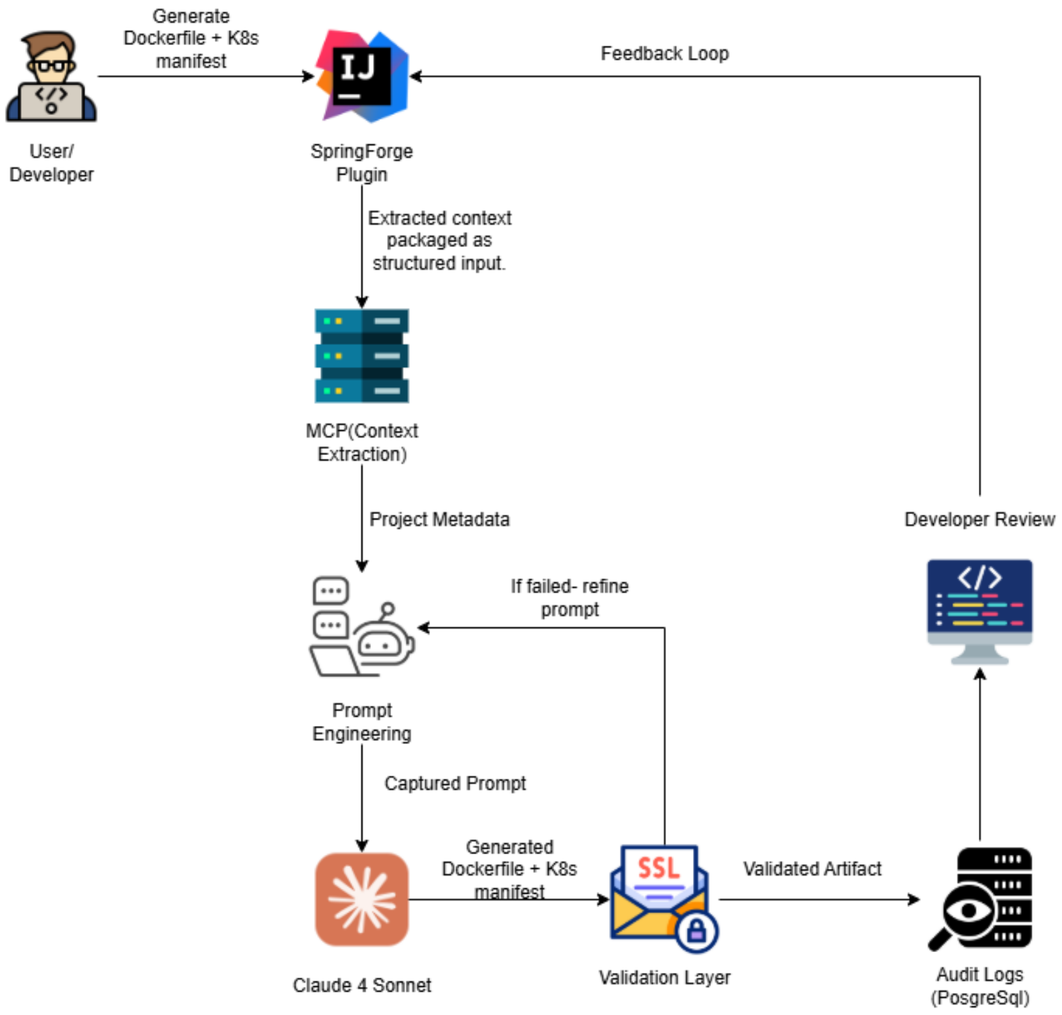
Data Sources:

- Internal Spring Boot projects and their configuration files.
- Best-practice guidelines from Docker, Kubernetes, and Spring Boot documentation.
- Logs from artifact generation, validation, and audit records.

Ethical Considerations:

- Anonymize project data and developer usage logs.
- Ensure Bedrock privacy guarantee → no model training on user data.
- Respect open-source licenses of validation rules/tools.
- Provide transparent explanations to prevent "blind trust" in AI-generated pipelines.

AI-Powered Infrastructure Generation with Security Validation and Explainability



A team building a large Spring Boot app uses the SpringForge Plugin to auto-generate secure Docker/K8s/CI pipelines with Claude 4, validated and explained for deployment.

- Developer requests CI/CD artifacts inside IntelliJ Plugin.
- MCP module extracts live project metadata(ports,dependencies, configs).
- Prompt Engineering structures this data and sends it to Claude 4 Sonnet (via Bedrock).
- Claude 4 generates Dockerfile, Kubernetes manifests, and pipeline definitions.
- Validation tools (Hadolint, KubeLinter, Spring Boot rules)automatically scan outputs.
- If issues are detected → feedback loop triggers Claude to refine artifacts.
- Validated artifacts are stored in audit logs and presented with line-by-line explanations.
- Developer reviews outputs, learns from explanations, and deploys with confidence

Commercialization Strategy

Value

- **Unified Lifecycle Support:** A single plugin for the entire development workflow.
- **Architecture-Aware Intelligence:** Prevents technical debt by understanding and enforcing project architecture.
- **Built-in DevSecOps:** Integrates security by default, not as an afterthought.

Target

- Enterprise Spring Boot Teams
- DevOps & Platform Engineers
- Junior Developers & Newcomers

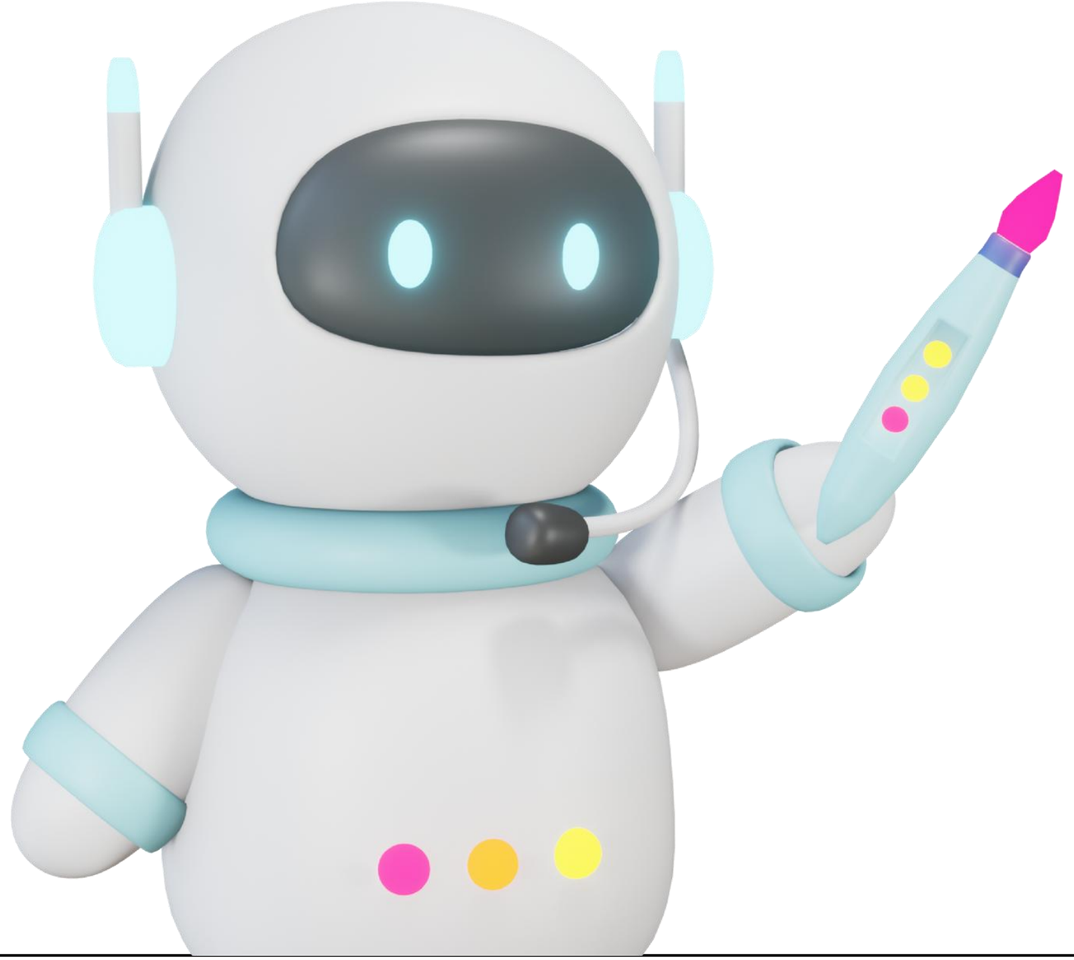
Investment

Cost

- Internet Charges: ~$20
- ML Model Hosting (Google Cloud Run):Free Tier
- LLM API Access (e.g., Amazon Bedrock): ~$30
- Total Estimated Operational Cost: ~$50

Investment

- Revenue Model: Freemium Subscription.
- Community: Free
- Professional and Enterprise editions with paid subscription with enhanced features



Thank You

Feature / Criteria	**Claude 4 Sonnet (Bedrock)**
Context Length	**200K+ tokens** (can handle full Spring Boot projects, configs)
Reasoning Ability	**Excellent** – strong at logical chains, code explanatio security reasoning
Security Awareness	**High** – designed with safety guardrails + good at DevSecOps validation